

EC-341 Digital System Design

Experiment # 2

Introduction to Xilinx Vivado

Objective/s:

- Xilinx ISE Design Tool
- Tasks

1. Xilinx ISE Design Tool

Xilinx Tools is a suite of software tools used for the design of digital circuits implemented using Xilinx Field Programmable Gate Array (FPGA) or Complex Programmable Logic Device (CPLD). The design procedure consists of (a) design entry, (b) synthesis and implementation of the design, (c) functional simulation and (d) testing and verification. Digital designs can be entered in various ways using the above CAD tools: using a schematic entry tool, using a hardware description language (HDL) – Verilog or VHDL or a combination of both. In this lab we will only use the design flow that involves the use of Verilog HDL.

The CAD tools enable you to design combinational and sequential circuits starting with Verilog HDL design specifications. The steps of this design procedure are listed below:

1. Create Verilog design input file(s) using template driven editor.
2. Compile and implement the Verilog design file(s).
3. Create the test-vectors and simulate the design (functional simulation) without using a PLD (FPGA or CPLD).
4. Assign input/output pins to implement the design on a target device.
5. Download bit stream to an FPGA or CPLD device.
6. Test design on FPGA/CPLD device

A Verilog input file in the Xilinx software environment consists of the following segments:

Header: module name, list of input and output ports.

Declarations: input and output ports, registers and wires.

Logic Descriptions: equations, state machines and logic functions.

End: endmodule

1.1 Creating Project

Xilinx Tools can be started by clicking on the Project Navigator Icon on the Windows desktop. This should open up the Project Navigator window on your screen. This window shows (see Figure 1) the last accessed project

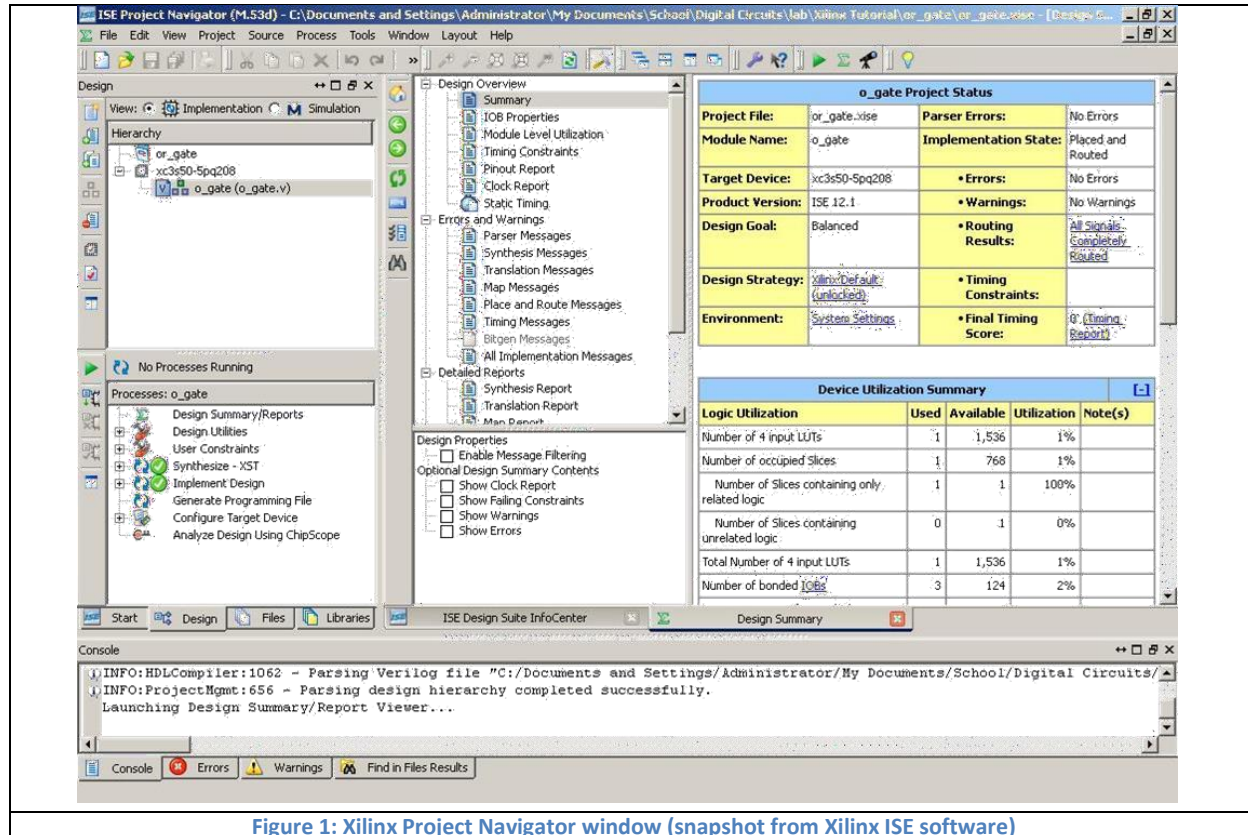


Figure 1: Xilinx Project Navigator window (snapshot from Xilinx ISE software)

Select **File->New Project** to create a new project. This will bring up a new project window (Figure 2) on the desktop. Fill up the necessary entries as follows:

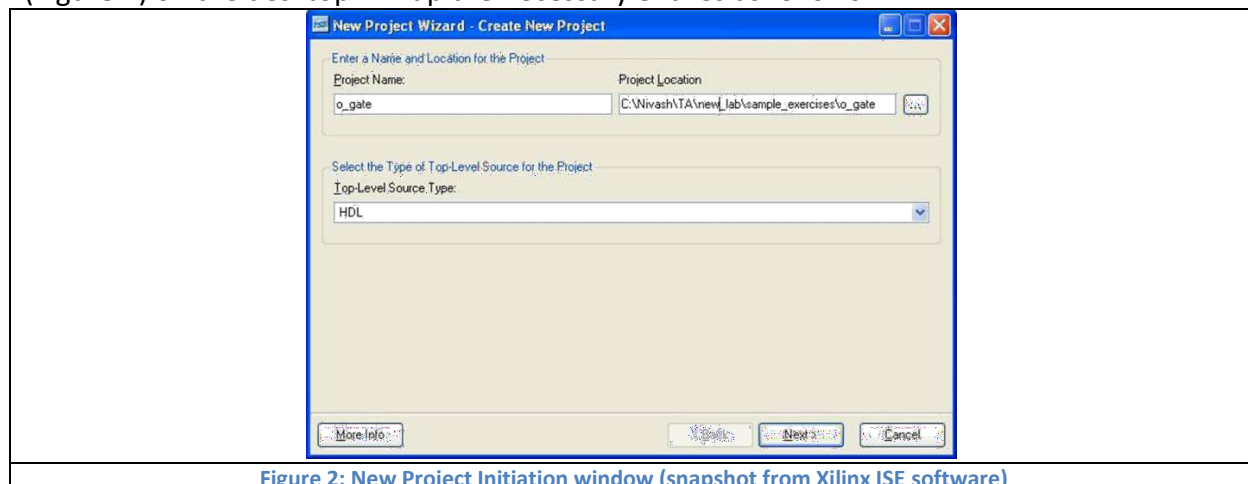


Figure 2: New Project Initiation window (snapshot from Xilinx ISE software)

- **Project Name:** Write the name of your new project
- **Project Location:** The directory where you want to store the new project (Note: DO NOT specify the project location as a folder on Desktop or a folder in the Xilinx\bin directory. Your H: drive is the best place to put it. **The project location path is NOT to have any spaces in it e.g.: C:\Nivash\TA\new lab\sample exercises\o_gate is NOT to be used**)

Leave the top level module type as HDL.

Example: If the project name were “o_gate”, enter “o_gate” as the project name and then click “Next”.

Clicking on NEXT should bring up the following window

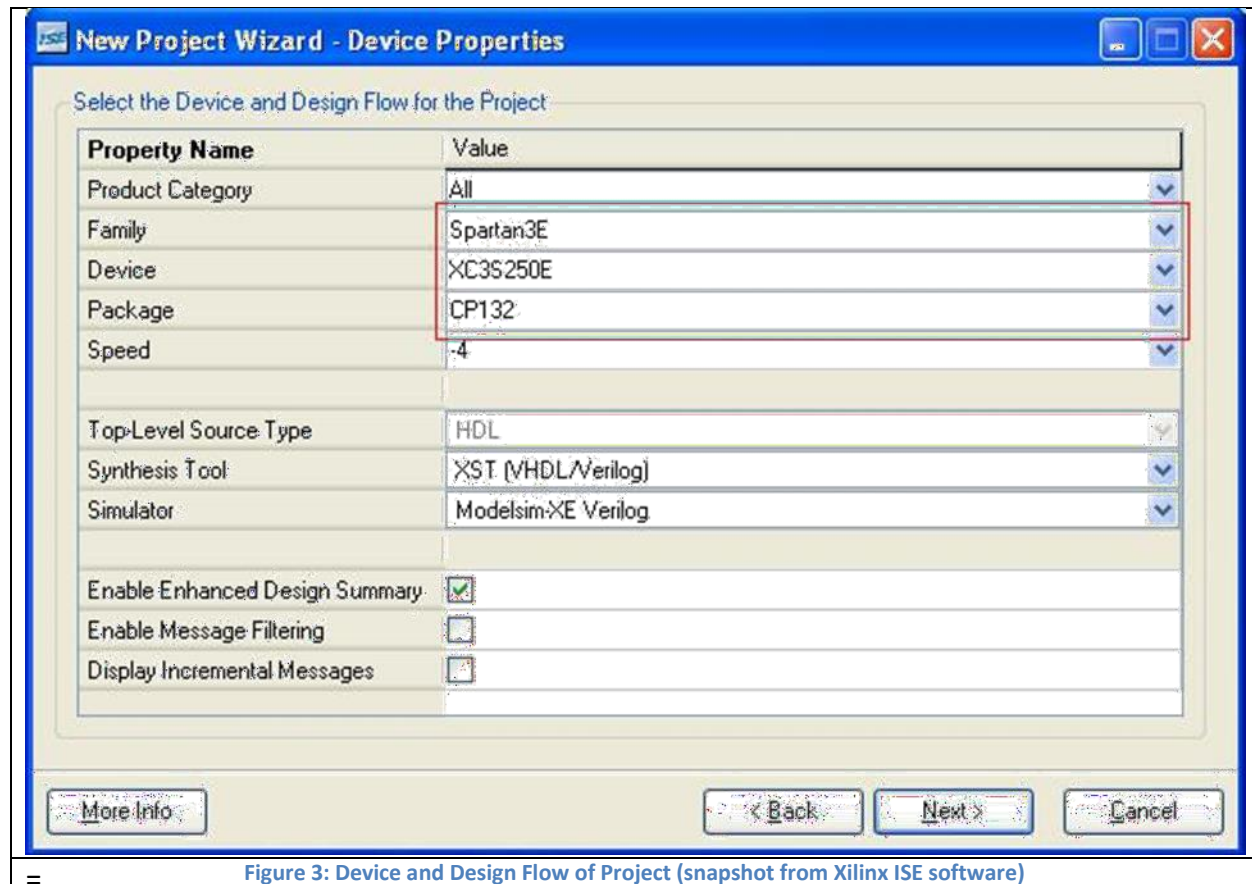


Figure 3: Device and Design Flow of Project (snapshot from Xilinx ISE software)

- For each of the properties given below, click on the ‘value’ area and select from the list of values that appear.

Device Family: Family of the FPGA/CPLD used. In this laboratory we will be using the Spartan3E FPGA’s.

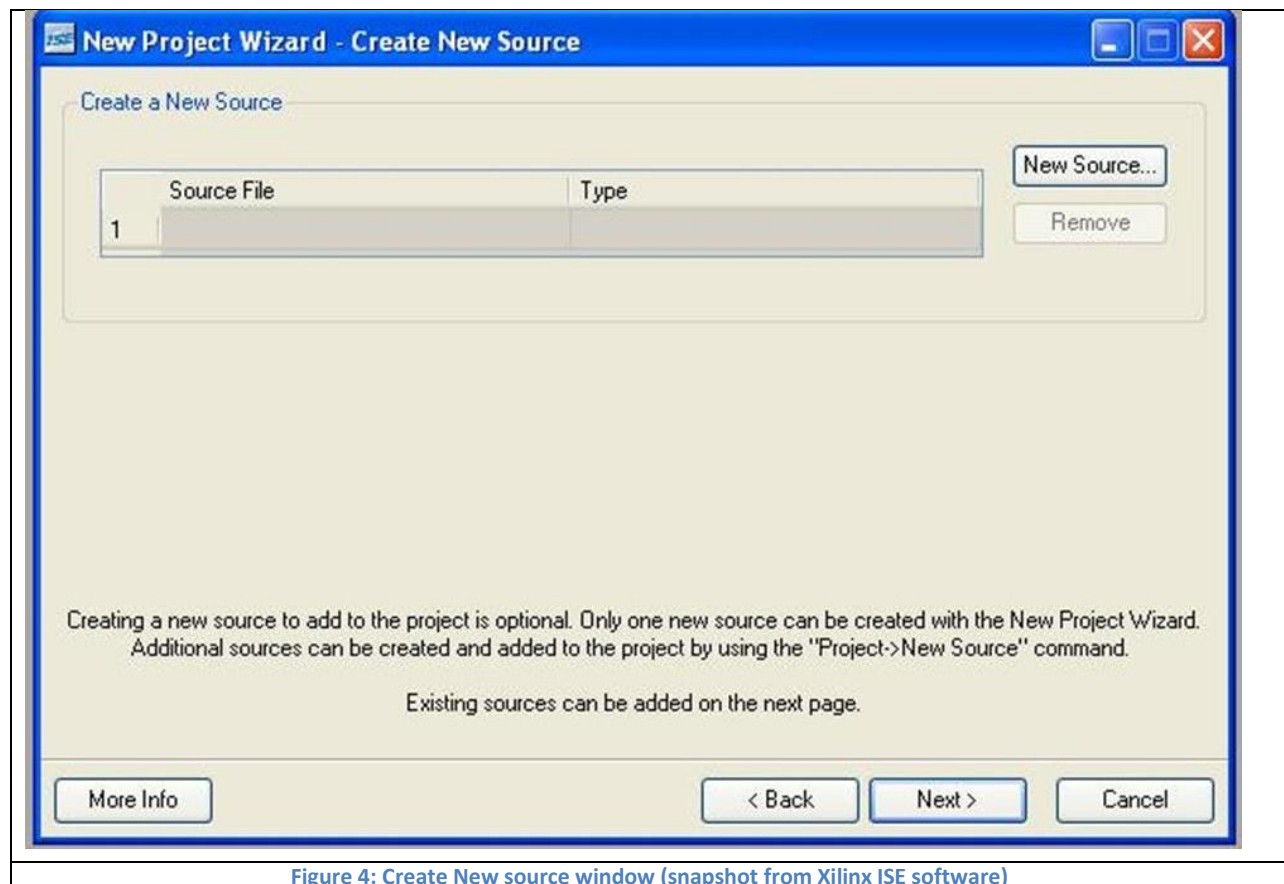
- **Device:** The number of the actual device. For this lab you may enter **XC3S250E** (this can be found on the attached prototyping board)
- **Package:** The type of package with the number of pins. The Spartan FPGA used in this lab is packaged in CP132 package.
- **Speed Grade:** The Speed grade is “-4”.
- **Synthesis Tool:** **XST** [VHDL/Verilog]
- **Simulator:** The tool used to simulate and verify the functionality of the design. Modelsim simulator is integrated in the Xilinx ISE. Hence choose “Modelsim-XE Verilog” as the simulator or even Xilinx ISE Simulator can be used.

Then click on **NEXT** to save the entries.

All project files such as schematics, netlists, Verilog files, VHDL files, etc., will be stored in a subdirectory with the project name. A project can only have one top level HDL source file (or schematic). Modules can be added to the project to create a modular, hierarchical design (see [Section 9](#)).

In order to open an existing project in Xilinx Tools, select **File->Open Project** to show the list of projects on the machine. Choose the project you want and click **OK**.

Clicking on NEXT on the above window brings up the following window:



If creating a new source file, Click on the NEW SOURCE.

In this lab we will enter a design using a structural or RTL description using the Verilog HDL. You can create a Verilog HDL input file (.v file) using the HDL Editor available in the Xilinx ISE Tools (or any text editor).

In the previous window, click on the NEW SOURCE

A window pops up as shown in [Figure 4](#). (Note: “**Add to project**” option is selected by default. If you do not select it then you will have to add the new source file to the project manually.)

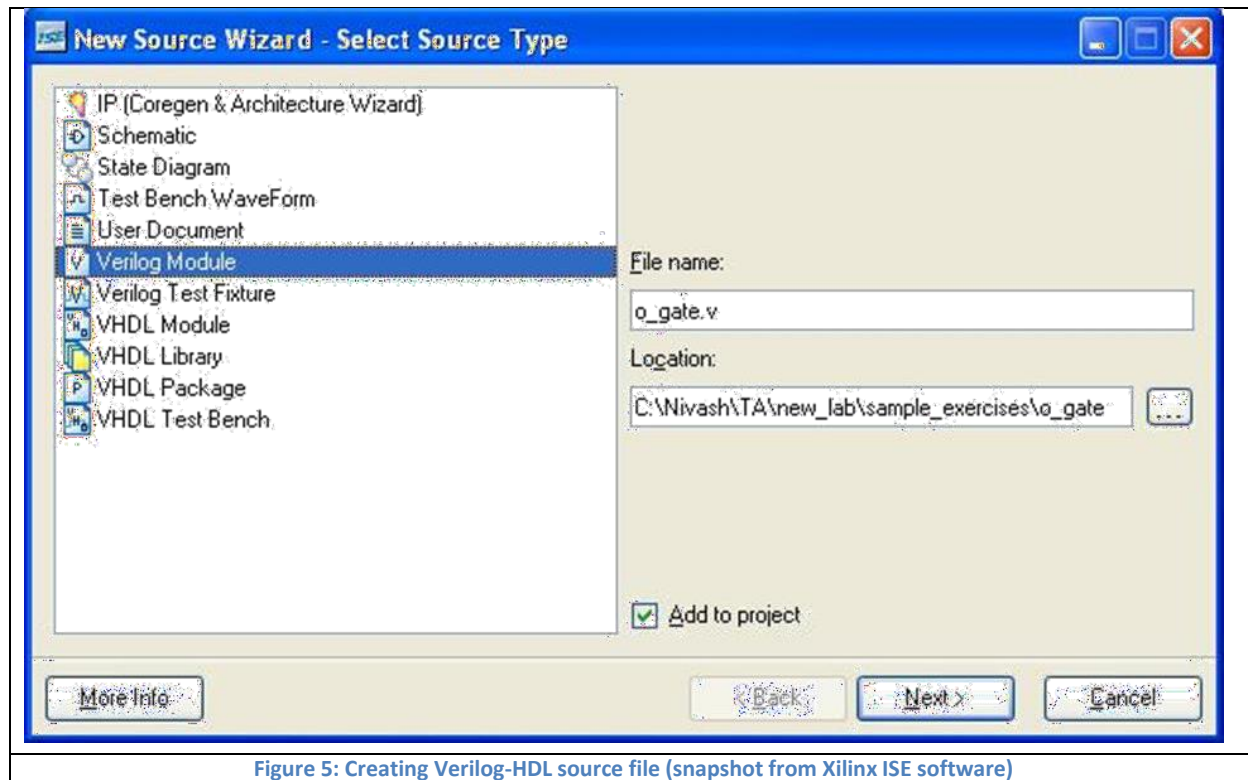


Figure 5: Creating Verilog-HDL source file (snapshot from Xilinx ISE software)

Select **Verilog Module** and in the “File Name:” area, enter the name of the Verilog source file you are going to create. Also make sure that the option **Add to project** is selected so that the source need not be added to the project again. Then click on **Next** to accept the entries. This pops up the following window ([Figure 5](#)).

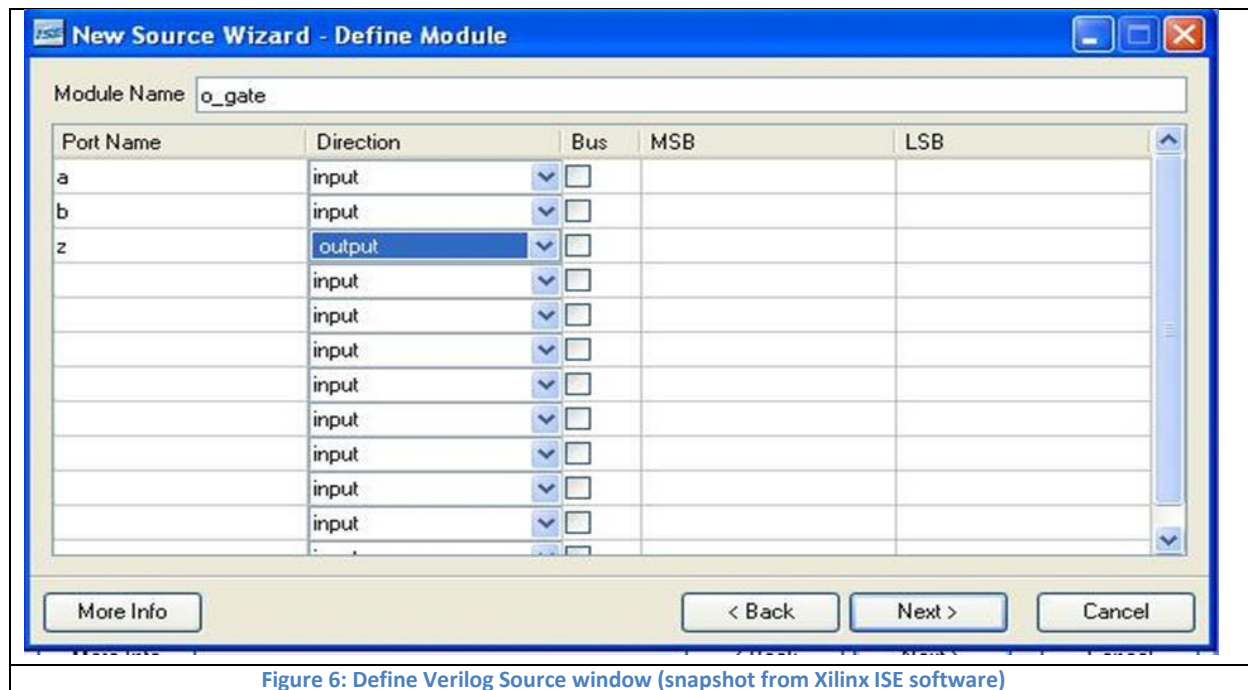


Figure 6: Define Verilog Source window (snapshot from Xilinx ISE software)

In the **Port Name** column, enter the names of all input and output pins and specify the **Direction** accordingly. A Vector/Bus can be defined by entering appropriate bit numbers in the **MSB/LSB** columns. Then click on **Next>** to get a window showing all the new source information (Figure 6). If any changes are to be made, just click on **<Back** to go back and make changes. If everything is acceptable, click on **Finish > Next > Next > Finish** to continue.

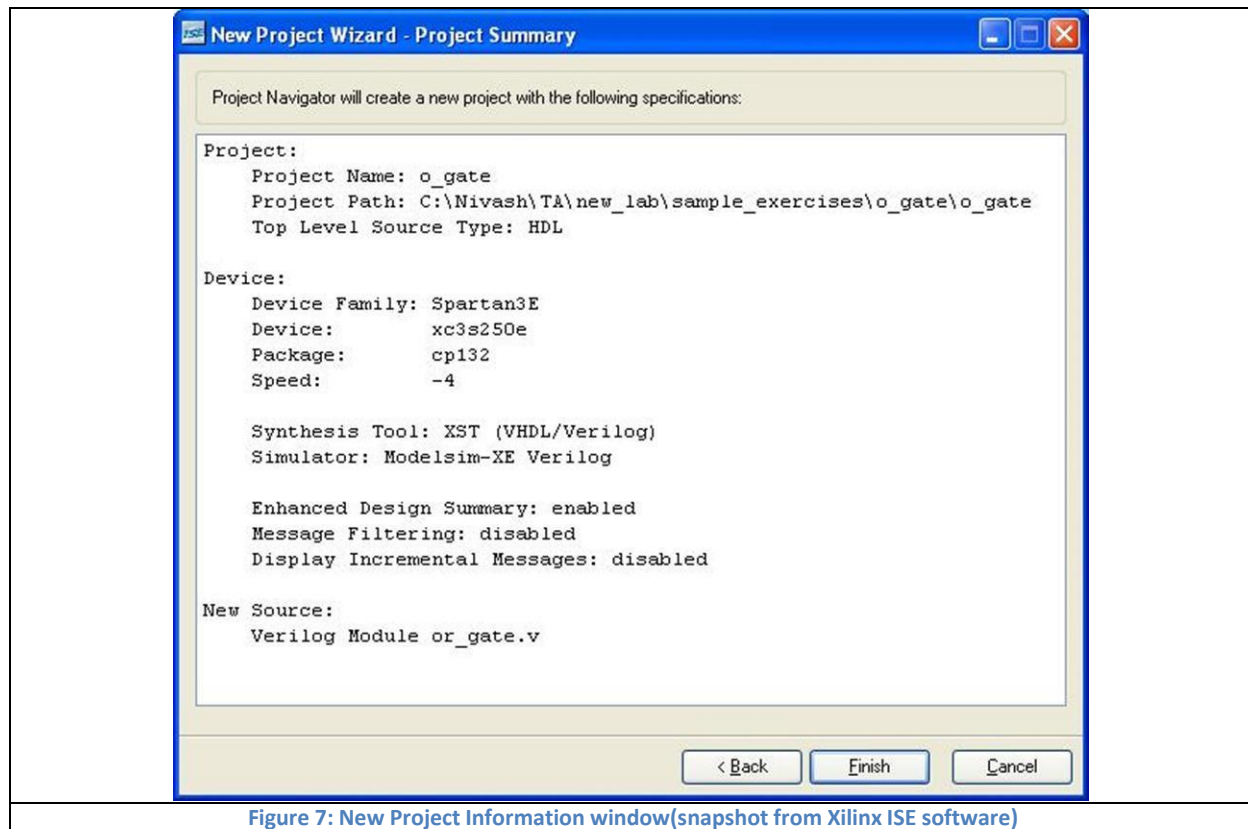


Figure 7: New Project Information window(snapshot from Xilinx ISE software)

Once you click on **Finish**, the source file will be displayed in the sources window in the **Project Navigator** (Figure 1).

If a source has to be removed, just right click on the source file in the **Sources in Project** window in the **Project Navigator** and select **Remove** in that. Then select **Project -> Delete Implementation Data** from the Project Navigator menu bar to remove any related files.

The source file will now be displayed in the **Project Navigator** window (Figure 8). The source file window can be used as a text editor to make any necessary changes to the source file. All the input/output pins will be displayed. Save your Verilog program periodically by selecting the **File->Save** from the menu. You can also edit Verilog programs in any text editor and add them to the project directory using "Add Copy Source".

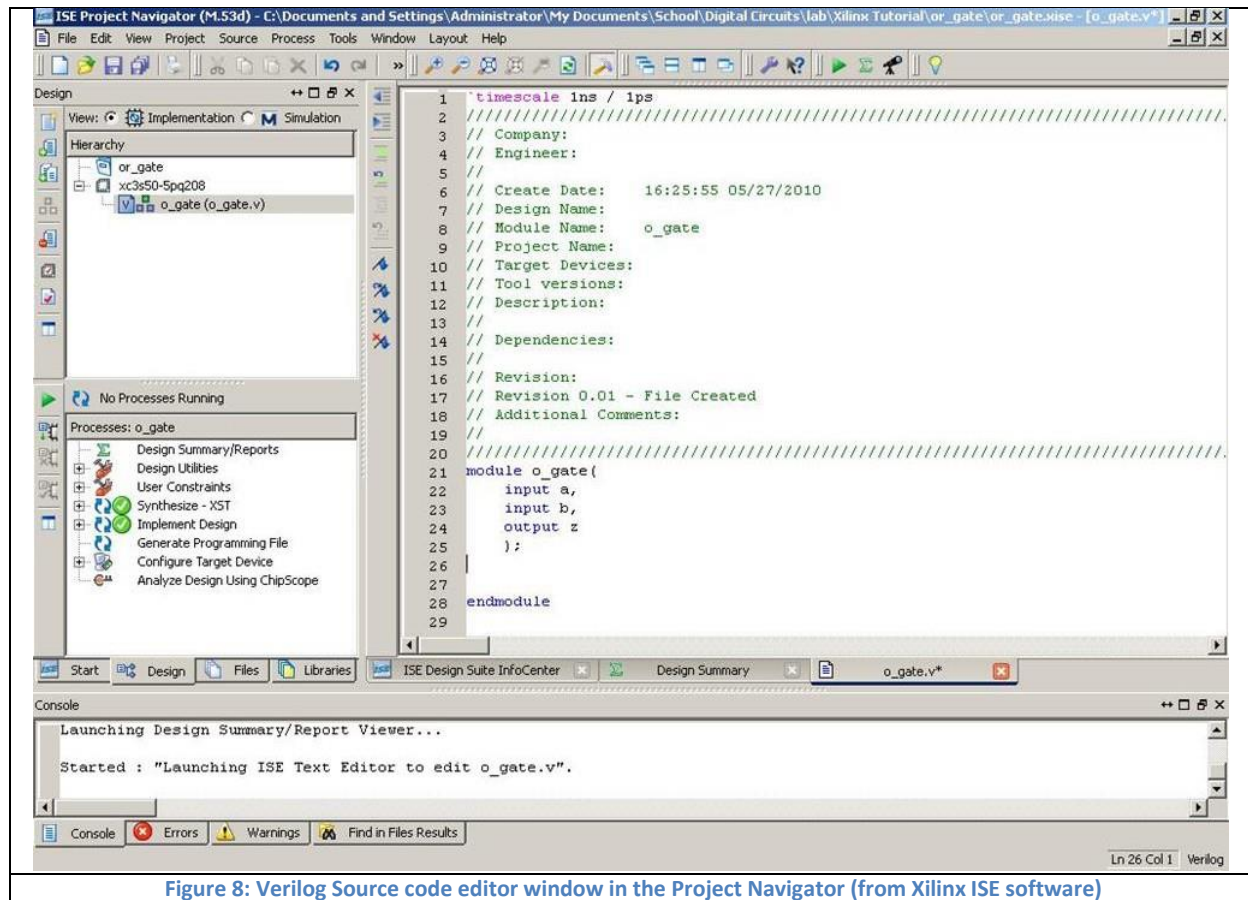


Figure 8: Verilog Source code editor window in the Project Navigator (from Xilinx ISE software)

- **Adding Logic in the generated Verilog Source code template:**

A brief Verilog Tutorial is available in Appendix-A. Hence, the language syntax and construction of logic equations can be referred to Appendix-A. The Verilog source code template generated shows the module name, the list of ports and also the declarations (input/output) for each port. Combinational logic code can be added to the verilog code after the declarations and before the endmodule line.

For example, an output **z** in an OR gate with inputs **a** and **b** can be described as, assign **z = a | b;**

Remember that the names are case sensitive.

- **Other constructs for modeling the logic function:**

A given logic function can be modeled in many ways in verilog. Here is another example in which the logic function, is implemented as a truth table using a case statement:

```
module or_gate(a,b,z);
input a;
input b;
output z;

reg z;
```

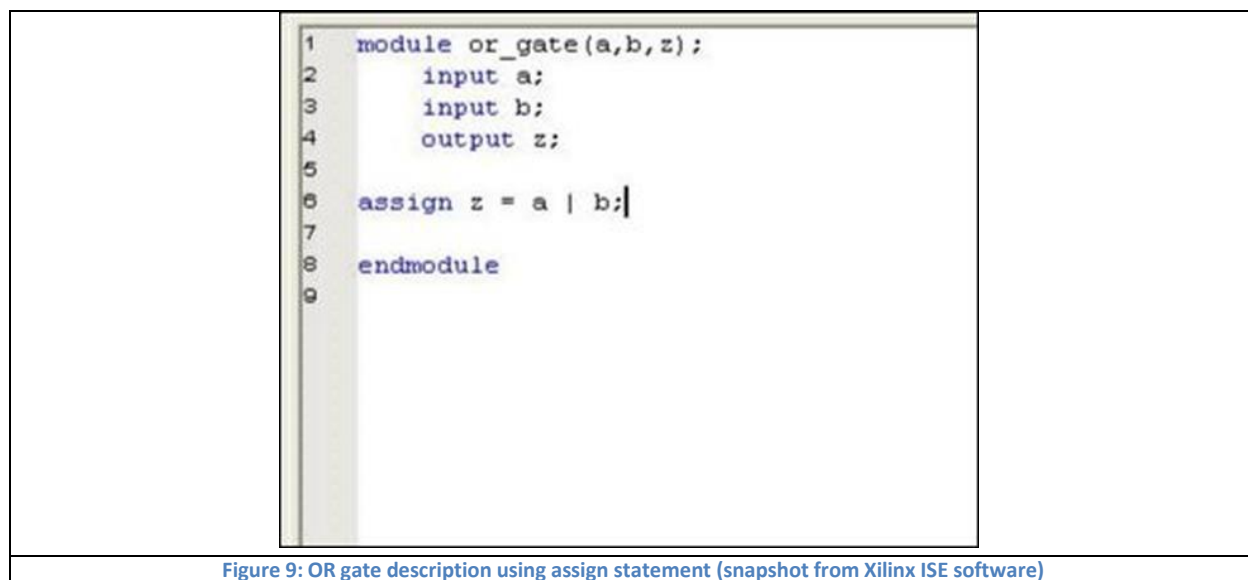
```
always @(a or b)
begin

    case ({a,b})

        0: z = 1'b0;
        1: z = 1'b1;
        10: z = 1'b1;
        11: z = 1'b1;
    endcase

end
endmodule
```

Suppose we want to describe an OR gate. It can be done using the logic equation as shown in [Figure 9a](#) or using the case statement (describing the truth table) as shown in [Figure 9b](#). These are just two example constructs to design a logic function. Verilog offers numerous such constructs to efficiently model designs. A brief tutorial of Verilog is available in Appendix-A.



The design has to be synthesized and implemented before it can be checked for correctness, by running functional simulation or downloaded onto the prototyping board. With the top-level Verilog file opened (can be done by double-clicking that file) in the HDL editor window in the right half of the Project Navigator, and the view of the project being in the **Module view**, the **implement design** option can be seen in the **process view**. **Design entry utilities** and **Generate Programming File** options can also be seen in the process view. The former can be used to include user constraints, if any and the latter will be discussed later.

To synthesize the design, double click on the **Synthesize Design** option in the **Processes window**.

To implement the design, double click the **Implement design** option in the **Processes window**. It will go through steps like **Translate, Map and Place & Route**. If any of these steps could not be done or done with errors, it will place a **X** mark in front of that, otherwise a tick mark will be placed after each of them to indicate the successful completion. If everything is done successfully, a tick mark will be placed before the **Implement Design** option. If there are warnings, one can see **!** mark in front of the option indicating that there are some warnings. One can look at the warnings or errors in the **Console** window present at the bottom of the Navigator window. **Every time the design file is saved; all these marks disappear asking for a fresh compilation.**

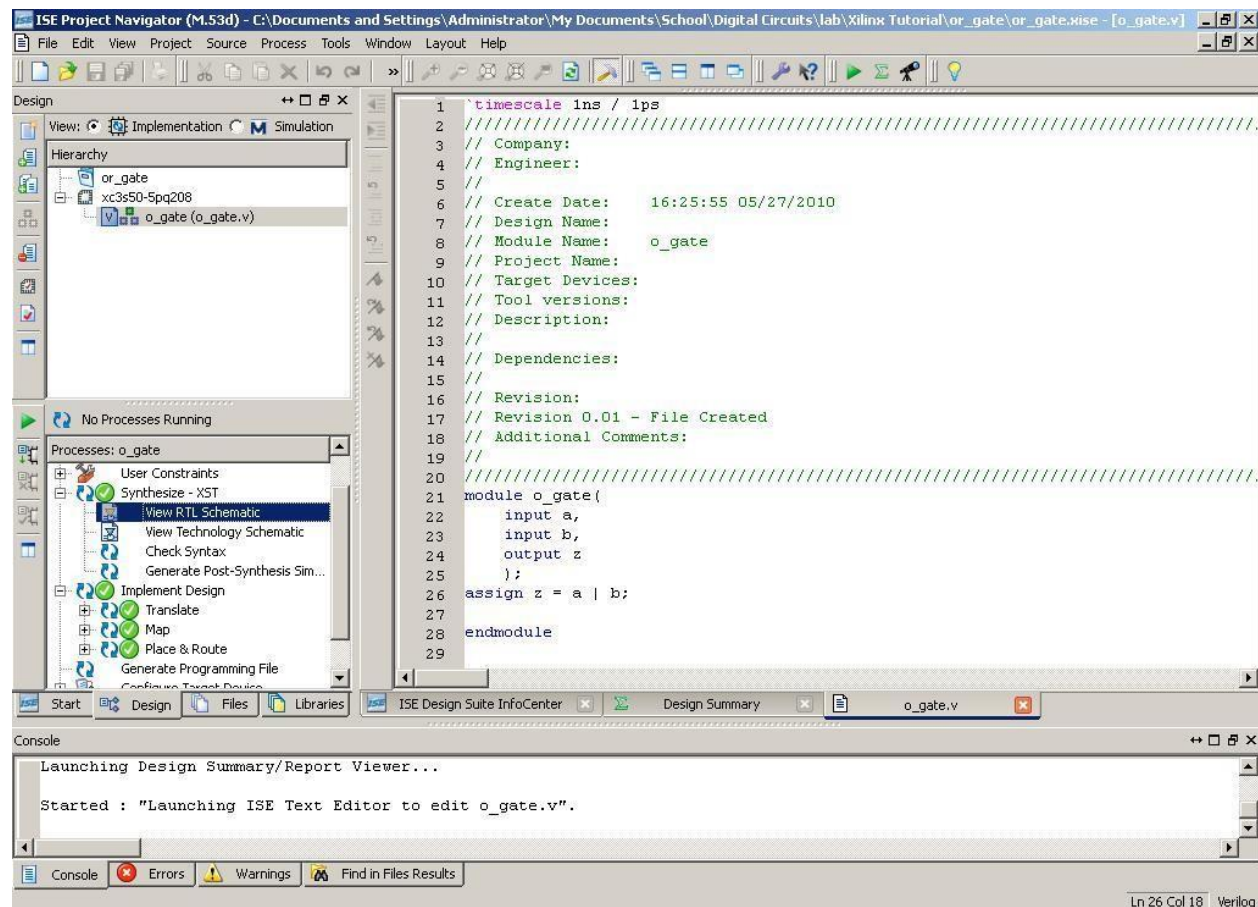


Figure 11: Implementing the Design (snapshot from Xilinx ISE software)

The schematic diagram of the synthesized verilog code can be viewed by double clicking View RTL Schematic under Synthesize-XST menu in the Process Window. This would be a handy way to debug the code if the output is not meeting our specifications in the proto type board. By double clicking it opens the top level module showing only input(s) and output(s) as shown below.

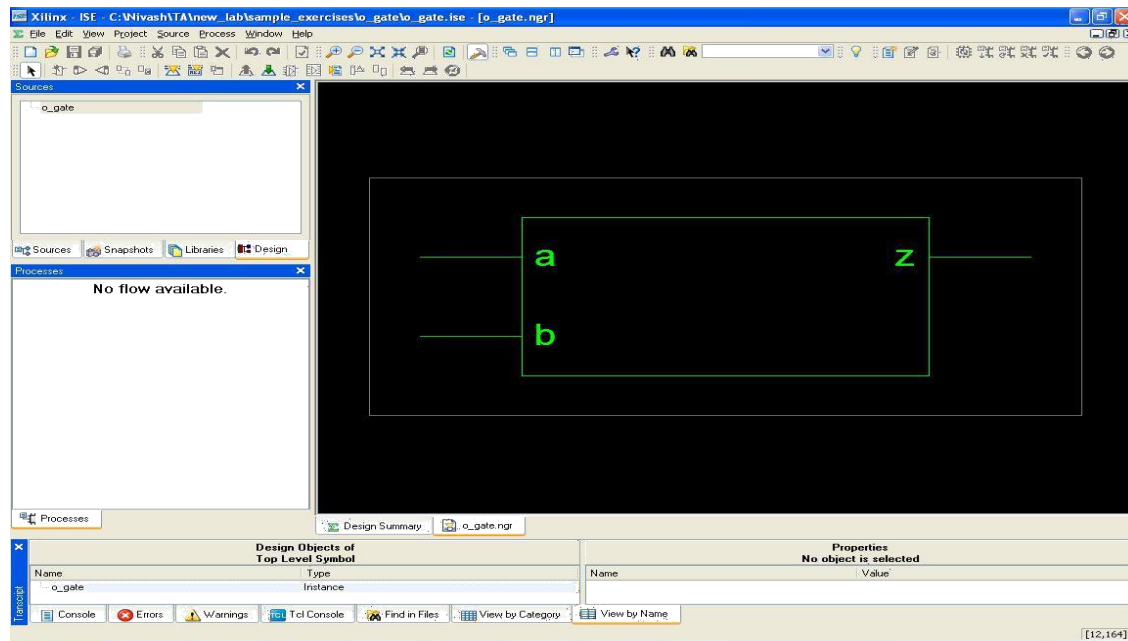
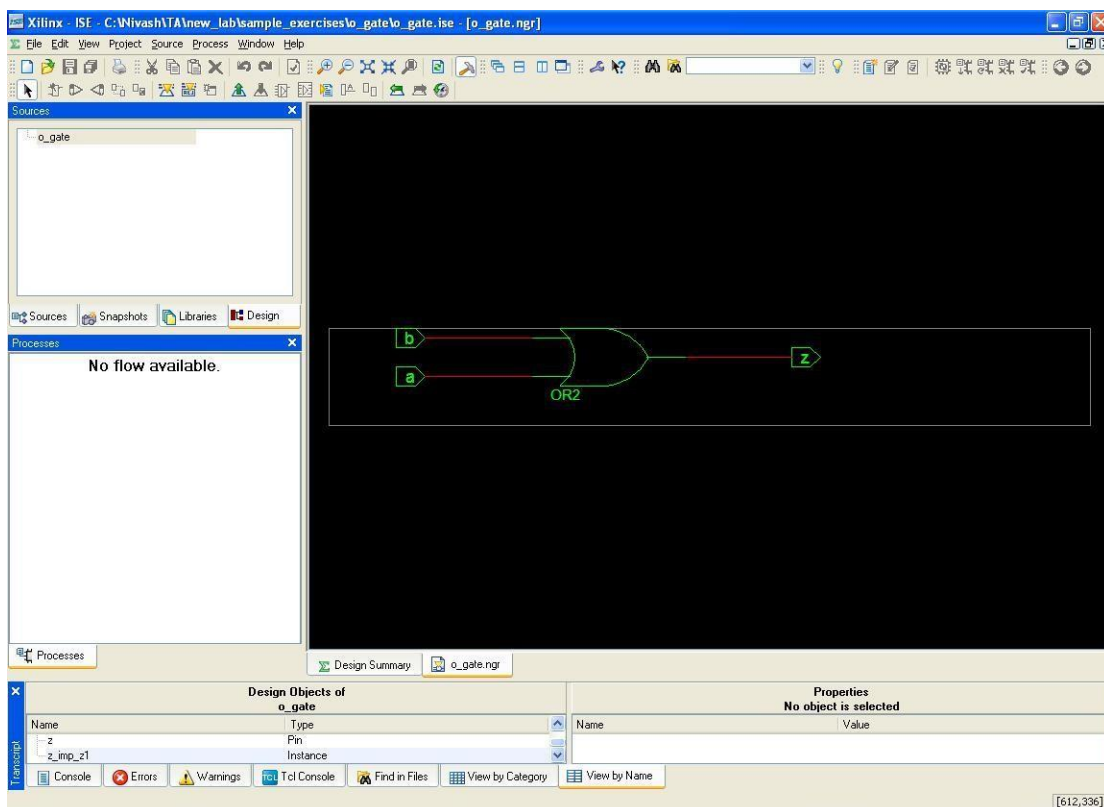


Figure 12: Top Level Hierarchy of the design

By double clicking the rectangle, it opens the realized internal logic as shown below.



To check the functionality of a design, we have to apply test vectors and simulate the circuit. In order to apply test vectors, a test bench file is written. Essentially it will supply all the inputs to the module designed and will check the outputs of the module. Example: For the 2 input OR Gate, the steps to generate the test bench is as follows:

In the **Sources window** (top left corner) right click on the file that you want to generate the test bench for and select **'New Source'**

Provide a name for the test bench in the file name text box and select **'Verilog test fixture'** among the file types in the list on the right side as shown in figure 11.

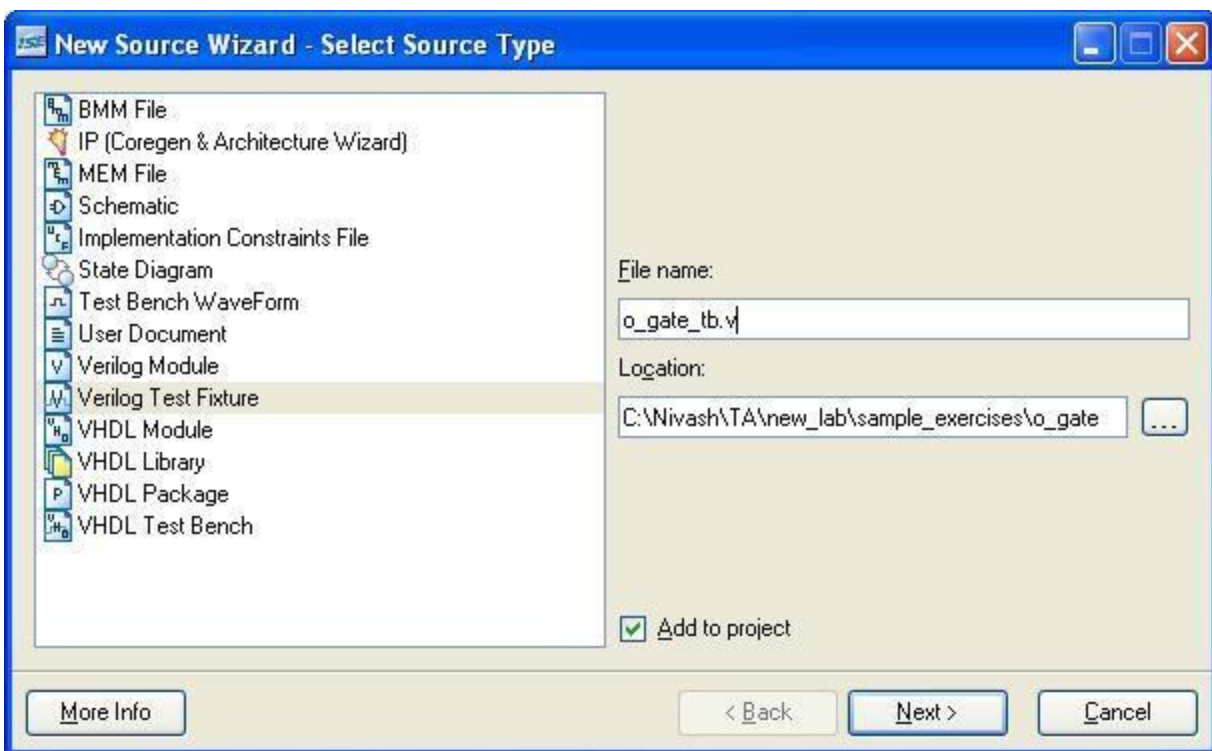


Figure 14: Adding test vectors to the design (snapshot from Xilinx ISE software)

Click on 'Next' to proceed. In the next window select the source file with which you want to associate the test bench.

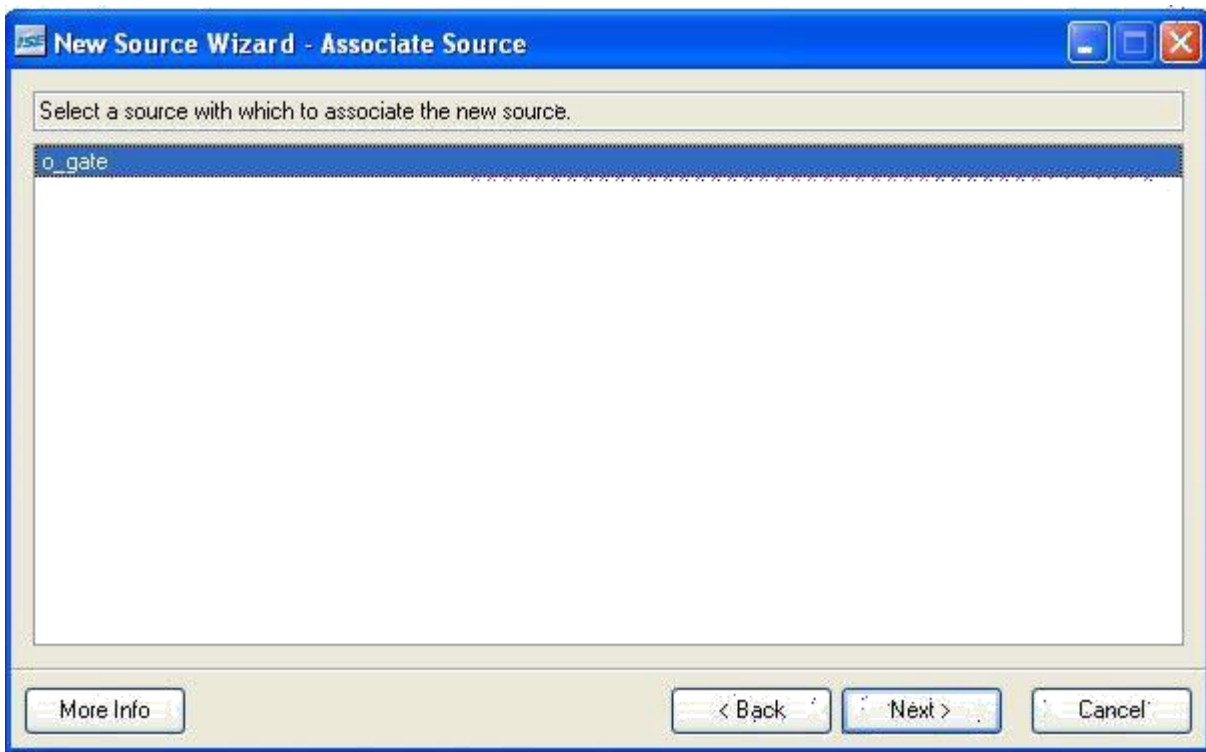
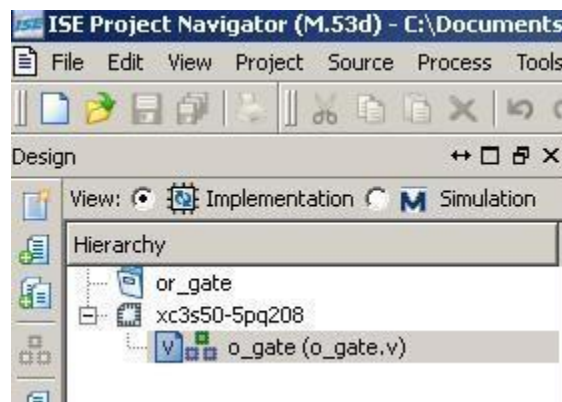


Figure 15: Associating a module to a testbench (snapshot from Xilinx ISE software)

Click on Next to proceed. In the next window click on Finish. You will now be provided with a template for your test bench. If it does not open automatically click the radio button next to **Simulation** .



Now under the **Processes window** (making sure that the testbench file in the **Sources window** is selected) expand the **ModelSim simulator Tab** by clicking on the add sign next to it. Double Click on **Simulate Behavioral Model**. You will probably receive a compiler error. This is nothing

to worry about – answer “No” when asked if you wish to abort simulation. This should cause ModelSim to open. Wait for it to complete execution. If you wish to not receive the compiler error, right click on **Simulate Behavioral Model** and select process properties. Mark the checkbox next to “Ignore Pre-Compiled Library Warning Check”.

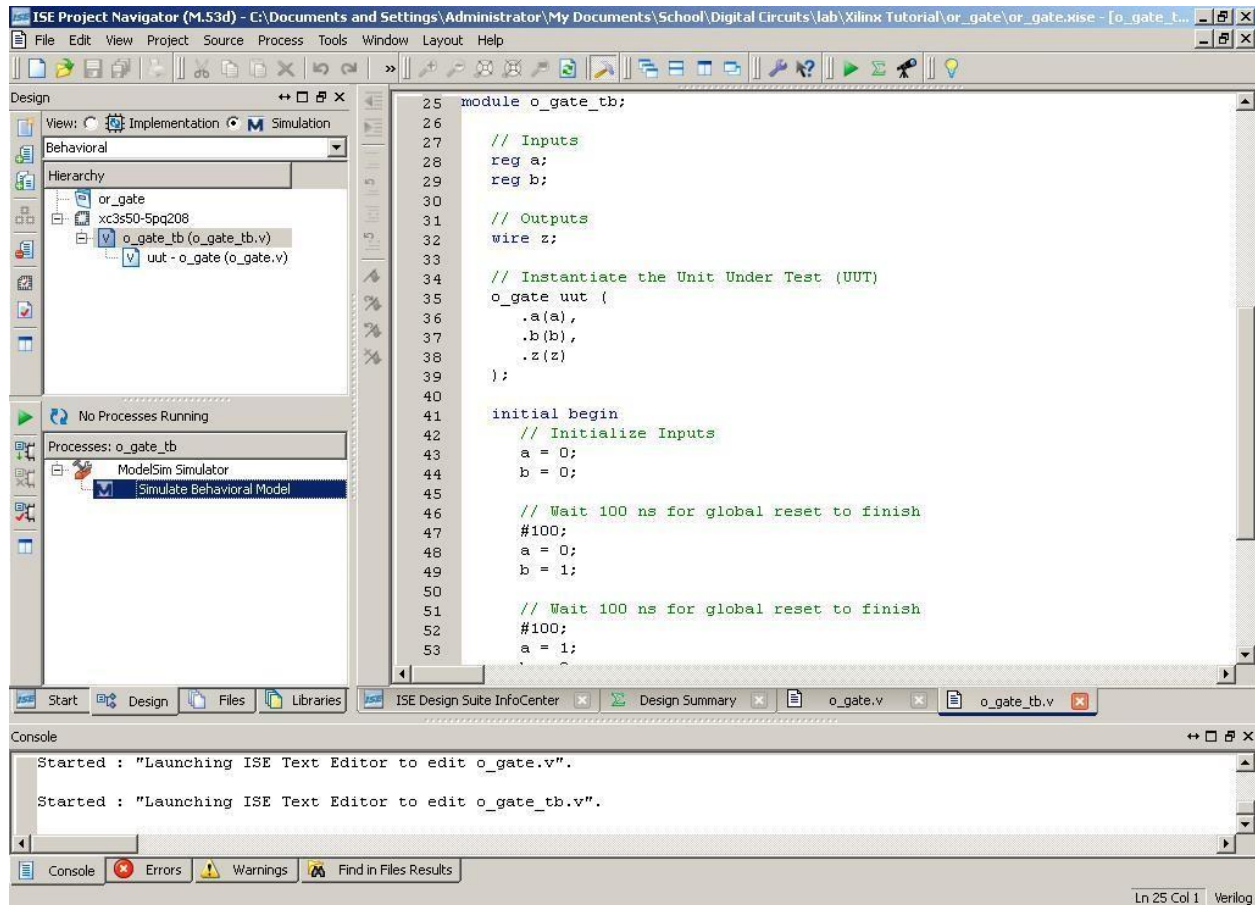


Figure 16: Simulating the design (snapshot from Xilinx ISE software)

To save the simulation results, Go to the waveform window of the Modelsim simulator, Click on File -> Print to Postscript -> give desired filename and location.

Note that by default, the waveform is “zoomed in” to the nanosecond level. Use the zoom controls to display the entire waveform.

Else a normal print screen option can be used on the waveform window and subsequently stored in Paint.

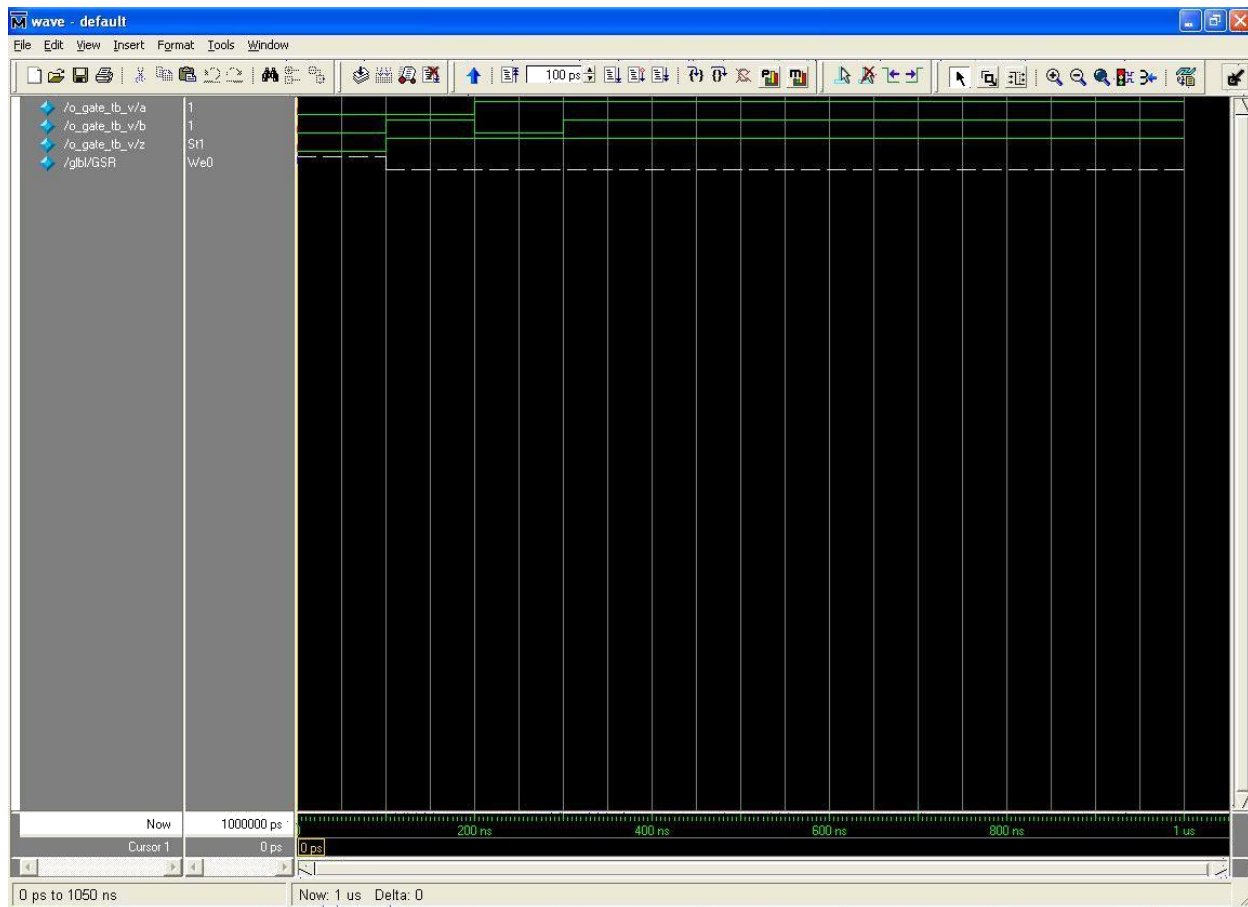


Figure 17: Behavioral Simulation output Waveform (Snapshot from ModelSim)

A bitstream file needs to be prepared for each design and downloaded onto the Basys2 prototyping board. This is done as follows:

- User Constraint File:
 - In order to test the design in the Basys2 board, the inputs need to be connected to the switches/buttons on the board and the outputs need to be connected to the onboard LED's.
 - To create the constraint file, ensure that the implementation radio button is selected and your verilog module is highlighted. In the processes window, expand User Constraints and double click on I/O Pin Planning (Plan Ahead) Post Synthesis. Answer "Yes" when asked if you want to create the UCF file. This will create the constraint file but also open the Plan Ahead application. It is beyond the scope of this tutorial to cover Plan Ahead. Wait for the Plan Ahead to fully open, then close it. You will now see the .ucf file in your hierarchy. Double click it to edit the file.
 - Assign pin numbers to the input and output pins in the Verilog design file using a "User Constraint File(ucf file)". The pin numbers can be assigned by looking at [section 7](#) of this tutorial. Then save the design file and implement the design again. **Note that you can assign pin numbers only to top-level Verilog file.** Then the Project Navigator window looks as shown in [Figure 13](#).

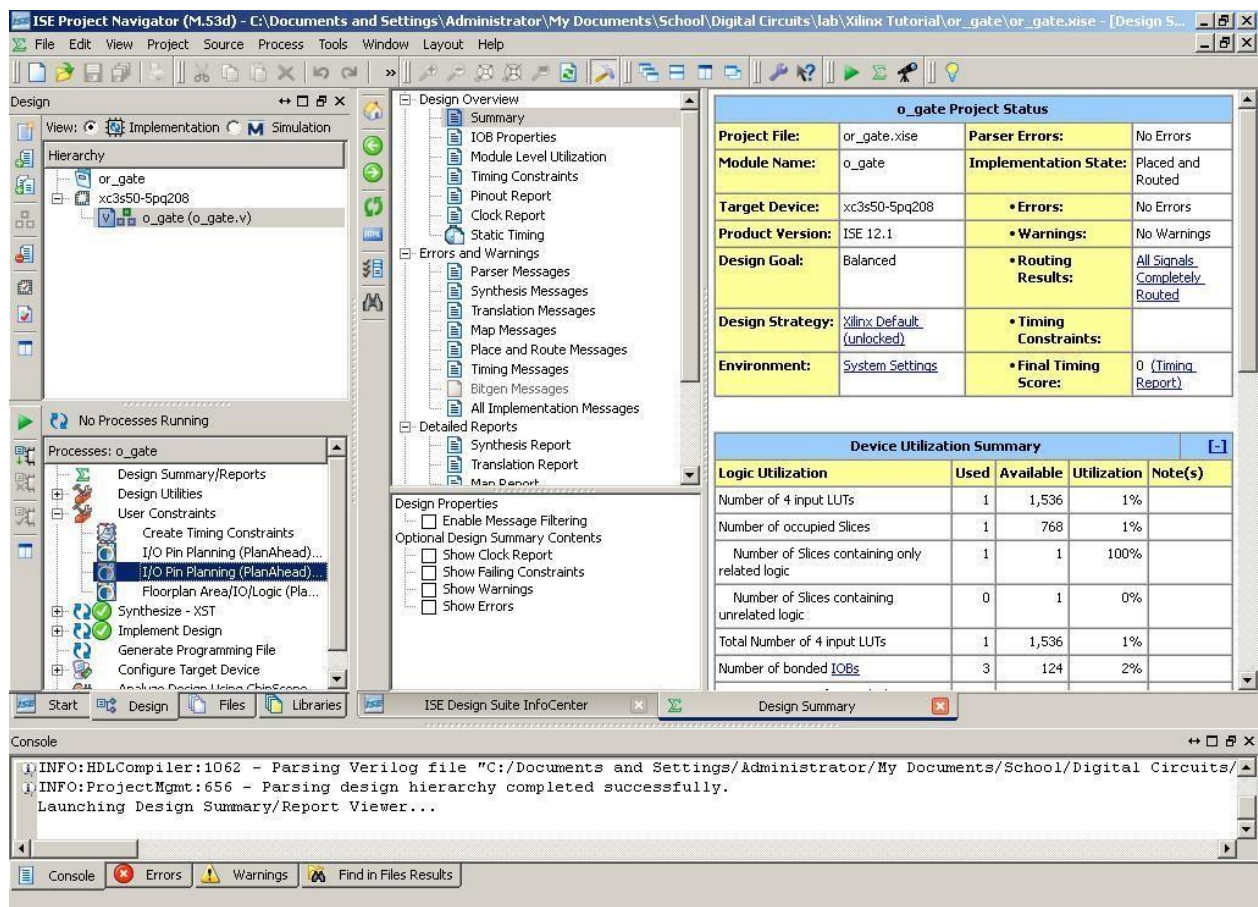


Figure 19: User Constraint File (snapshot from Xilinx ISE software)

For the OR_GATE example, the user constraint file used is as follows:

#Balls P11 and L3 of FPGA are connected to SW0 and SW1 in Basys2 Board and Ball M5 of FPGA is connected to LED0 in Basys2 Board.

```
# OR Gate Constraint list - Comment
NET a LOC = "P11";
NET b LOC = "L3";
NET z LOC = "M5";
```

In the **Sources View**, choose the main design file and in the Process View -> **User Constraints** -> Edit Constraints option, add the user constraint file for the design.

- Then, in the **Process View** window (left-bottom), double click on the 'Generate Programming file'.

2. Tasks

1. Write a Gate Level Verilog code to find 1's complement of three bit numbers.
2. *Write* a Gate Level Verilog code for design and test bench of Half Adder in Xilinx Vivado. Simulate the Adder for different set of inputs. Attach waveform screenshots, schematic and Verilog code.
3. *Write* code for above task for Full Adder as well.